

Airflow

28 December 2025

23:41

→ An orchestrator: schedule tasks in right way & organizes & combines them.

```
python
from airflow.decorators import dag, task

@dag(start_date=datetime(2023, 1, 1),
      schedule='@daily',
      description='my dag does that',
      tags=['team-a'])
def my_dag:
    @task
    def extract():
        return 42

    @task
    def transform(val):
        return val+42

    val = extract()
    new_val = transform(val)
```

Can also be created @ runtime + from yaml files.

- easy to scale number of tasks.
 - can run on celery, kubernetes, dask, etc.
 - Pipeline needs to be a direct acyclic graph (DAG)
 - you can use operators to do specific tasks (eg: pythonOperator, s3KeySensor, etc.)
- These are just tasks.
@dag is the python operator behind the scenes.
- DAG run → instance of a dag at a specific time.

* Internals of Airflow



- * Airflow tells other systems when & in what order to run.
- * Celery runs the tasks as soon as possible, in the background.

↳ The function only runs when:

1. The DAG is triggered
2. The scheduler schedules the task
3. A worker (local, celery, or kubernetes executor) picks it up.

Execution happens

1. In a separate process,
2. Possibly on a different machine
3. With no shared memory w/ the scheduler or other tasks.

if you want horizontal scaling

- * If your task fails, the whole function is rerun
 - No checkpointing
 - No partial recovery
- } Spark on the other hand would have task level retries

Airflow tasks need to be bounded... because if task fails, it will be rerun (whole DAG won't be, just that task).

You'd use celery alone when tasks are event driven rather than scheduled.